Rôle de chaque fichier

Transaction Orchestrator — MOSI Fintech

Structure Spring Boot · 58 fichiers Java + 3 fichiers Docker

Principe de lecture

Chaque fichier dans ce projet a UN rôle précis et ne fait QU'UNE chose.

Ce document explique ce rôle en lien direct avec la tâche du Transaction Orchestrator.

Types : @Entity = table DB | @Service = logique métier | @Component = composant Spring | Interface = contrat

01 Point d'entrée de l'application

Fichier	Type	Rôle dans le Transaction Orchestrator
`TransactionOrchestratorApplication.java`	`@SpringBootApplication`	Point d'entrée unique du microservice. Lance le serveur Spring Boot, active le scan des composants, active le scheduling (@EnableScheduling pour le job de recovery toutes les 30s) et active le JPA.

02 Configuration — config/

Rôle du dossier config/

Contient les classes de configuration Spring Boot.

Ces classes ne contiennent pas de logique métier — elles configurent les connexions aux infrastructures externes.

Fichier	Type	Rôle dans le Transaction Orchestrator
`KafkaConfig.java`	`@Configuration`	Configure le producer Kafka (Outbox relay) et le consumer Kafka (fraud-alerts-service). Définit les topics mosi.transactions.* et mosi.fraud.signals. Configure la sérialisation JSON des messages.
`RedisConfig.java`	`@Configuration`	Configure la connexion Redis utilisée par IdempotencyService. Redis stocke temporairement les clés d'idempotence en cache pour accélerer la vérification anti double-clic (lecture en mémoire plutôt qu'en DB).
`GrpcConfig.java`	`@Configuration`	Configure les canaux gRPC vers wallet-service et ledger-service. Définit les adresses, ports, timeouts (3000ms), et les intercepteurs de sécurité pour les appels gRPC critiques.

03 Entités JPA — domain/entity/

Rôle du dossier domain/entity/

Chaque fichier représente une table PostgreSQL. JPA génère automatiquement les tables à partir de ces classes.

Ces classes sont les objets manipulés partout dans le code — du controller jusqu'au repository.

Fichier	Type	Rôle dans le Transaction Orchestrator
`Saga.java`	`@Entity`	Représente la table sagas. C'est l'objet central du microservice — une instance = une transaction complète (P2P, CASH_IN, etc.). Contient le status, les montants, les snapshots KYC/pricing/risk, la politique de remboursement et le version pour l'optimistic locking.
`SagaStep.java`	`@Entity`	Représente la table saga_steps. Une instance = une étape d'exécution d'une saga (ex : étape 7 WALLET_DEBIT). Contient le plan d'action (action_payload) ET le plan de rollback (compensating_action). C'est ici que l'orchestrator sait quoi faire et comment défaire.
`SagaAuditEvent.java`	`@Entity`	Représente la table saga_audit_events. Journal immuable append-only. Une instance = un événement qui s'est produit (SAGA_CREATED, STEP_FAILED, etc.). Ne peut jamais être modifié ni supprimé — règle PostgreSQL. Requis par la BEAC.
`OutboxEvent.java`	`@Entity`	Représente la table outbox_events. Une instance = un event Kafka à publier. Écrit dans la même transaction que la saga (atomicité garantie). Le OutboxRelayJob lit ces lignes et les publie sur Kafka. Sans ce fichier, des events peuvent être perdus si Kafka est down.
`IdempotencyKey.java`	`@Entity`	Représente la table idempotency_keys. Une instance = une clé de protection anti double-exécution. Si l'app mobile envoie 2 fois la même requête, la deuxième est ignorée et le résultat caché est renvoyé. Protège contre les doubles débits.
`CompensationLog.java`	`@Entity`	Représente la table compensation_log. Une instance = une action de rollback exécutée. Trace granulaire de chaque remboursement automatique. Si le rollback échoue 10 fois, passe en MANUAL_REQUIRED pour intervention du Finance Officer.

04 Enums — domain/enums/

Rôle du dossier domain/enums/

Les enums définissent toutes les valeurs possibles des champs statut, type, action.

Empêchent l'insertion de valeurs invalides en DB — double protection avec les contraintes CHECK PostgreSQL.

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaType.java`	`Enum`	Valeurs : P2P, P2B, CASH_IN, CASH_OUT, REFUND_P2P, REBALANCING. Détermine quel moteur d'exécution utiliser, combien d'étapes la saga a, et quels services appeler. Premier attribut lu par le SagaExecutor.
`SagaStatus.java`	`Enum`	Valeurs : PENDING, PROCESSING, COMPLETED, FAILED, COMPENSATION_IN_PROGRESS, COMPENSATION_COMPLETED, COMPENSATION_FAILED. Représente l'état global de la transaction. L'attribut le plus lu de toute la DB.
`StepStatus.java`	`Enum`	Valeurs : PENDING, EXECUTING, COMPLETED, FAILED, COMPENSATED, SKIPPED. COMPENSATED = étape réussie puis annulée lors d'un rollback. SKIPPED = jamais exécutée car saga échouée avant.
`ActionType.java`	`Enum`	Valeurs : HTTP_GET, HTTP_POST, GRPC, KAFKA_PUBLISH. Indique à l'orchestrator comment construire l'appel réseau vers chaque service cible lors de l'exécution d'une étape.
`CompensationType.java`	`Enum`	Valeurs : WALLET_CREDIT, WALLET_DEBIT, LEDGER_REVERSAL, COMMISSION_CANCEL, FLOAT_ADJUST, CASHLOG_VOID. Indique quel type d'action inverse a été exécutée lors du rollback.

05 Repositories JPA — repository/

Rôle du dossier repository/

Chaque interface étend JpaRepository — Spring génère automatiquement les requêtes SQL standards.

On y ajoute des requêtes personnalisées avec @Query pour les besoins spécifiques du Transaction Orchestrator.

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaRepository.java`	Interface JPA	Accès DB pour la table sagas. Contient des requêtes personnalisées : findByIdempotencyKey(), findByStatusAndUpdatedAtBefore() (pour le job de recovery qui détecte les sagas bloquées depuis plus de 5 minutes).
`SagaStepRepository.java`	Interface JPA	Accès DB pour la table saga_steps. Contient : findBySagaIdOrderByStepOrder() pour récupérer les étapes dans l'ordre, findBySagaIdAndStatus() pour récupérer les étapes à compenser lors d'un rollback.
`SagaAuditEventRepository.java`	Interface JPA	Accès DB pour saga_audit_events. Uniquement des opérations INSERT (save). Jamais de UPDATE ni DELETE — cohérent avec l'immuabilité de la table. Contient findBySagaIdOrderByCreatedAt() pour l'audit BEAC.
`OutboxEventRepository.java`	Interface JPA	Accès DB pour outbox_events. Contient findByPublishedAtIsNullAndPublishAttemptsLessThan() — requête utilisée par OutboxRelayJob toutes les secondes pour trouver les events non publiés sur Kafka.
`IdempotencyKeyRepository.java`	Interface JPA	Accès DB pour idempotency_keys. Contient findByKeyAndExpiresAtAfter() — vérifie si une clé existe et n'est pas expirée. C'est la première requête exécutée à chaque nouvelle demande de transaction.
`CompensationLogRepository.java`	Interface JPA	Accès DB pour compensation_log. Contient findByStatusAndNextRetryAtBefore() pour le job de recovery qui relance les compensations en attente, et findByStatusManualRequired() pour le dashboard du Finance Officer.

06 Services métier — service/

Rôle du dossier service/

Contient la logique métier pure du Transaction Orchestrator.

Les services sont appelés par le Controller et le SagaExecutor.

Chaque service a une responsabilité unique et précise.

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaService.java`	`@Service`	Orchestre la création et le suivi des sagas. Reçoit la demande du Controller, vérifie l'idempotence, crée la Saga en DB, délègue l'exécution au SagaExecutor. C'est le chef d'orchestre principal — il coordonne tous les autres services.
`CompensationService.java`	`@Service`	Gère tout le processus de rollback. Quand une étape échoue, ce service détermine quelles étapes doivent être compensées (dans l'ordre inverse), crée les lignes dans compensation_log, et exécute les appels de rollback vers les services concernés.
`IdempotencyService.java`	`@Service`	Gère la protection anti double-exécution. Vérifie si une clé existe (Redis d'abord, DB ensuite), crée la clé dans la même transaction que la saga, met à jour le result après completion. Garantit qu'une clé = une seule saga créée.
`AuditService.java`	`@Service`	Crée les événements d'audit immuables dans saga_audit_events. Appelé à chaque changement d'état important : création saga, démarrage étape, échec étape, démarrage compensation, etc. Inclut le trace_id OpenTelemetry pour la traçabilité distribuée.

07 Moteur d'exécution — executor/

Rôle du dossier executor/

C'est le coeur technique du Transaction Orchestrator.
Le SagaExecutor lit les étapes et les exécute dans l'ordre en appelant les bons services.
Chaque step est un composant indépendant qui implémente l'interface SagaStep.

executor/ — Moteur principal

Fichier	Type	Rôle dans le Transaction Orchestrator
SagaExecutor.java	@Component	Moteur principal d'exécution. Lit la liste des étapes d'une saga, les exécute dans l'ordre (1→10), gère les retry avec backoff exponentiel, détecte les échecs et déclenche la compensation via CompensationService. C'est lui qui donne vie à la saga.

executor/steps/ — Les étapes

Fichier	Type	Rôle dans le Transaction Orchestrator
SagaStep.java	Interface	Contrat que toutes les étapes doivent respecter. Définit la méthode execute(SagaContext) qui retourne un StepResult. Permet au SagaExecutor de traiter toutes les étapes de façon uniforme sans connaître leurs détails.
AccountCheckStep.java	@Component	Étape 1 — tous types. Appelle account-control-service via REST GET pour vérifier que les comptes sender et receiver sont actifs, non gelés, non suspendus. Si KO → saga FAILED sans compensation (rien de financier n'a été fait).
LimitsCheckStep.java	@Component	Étape 2 — tous types. Appelle limits-thresholds-service via REST POST pour vérifier que le montant respecte les plafonds journaliers et mensuels selon le profil KYC de l'utilisateur. Si KO → saga FAILED.
KycCheckStep.java	@Component	Étape 3 — tous types. Appelle kyc-kyb-orchestrator via REST GET pour vérifier le niveau KYC. Fige le résultat dans kyc_snapshot (immuable). Si KYC niveau 1 et montant > 100 000 XOF → saga FAILED.
PinVerifyStep.java	@Component	Étape 4 — P2P, P2B, CASH_OUT. Appelle pin-service via REST POST. max_retries=1 (anti-bruteforce). Non exécuté pour CASH_IN car l'agent physique valide l'identité en personne.
PricingStep.java	@Component	Étape 5 — tous types. Appelle pricing-service via REST POST. Calcule les frais (1% MOSI). Fige le résultat dans pricing_snapshot ET dans fees_cents/net_amount_cents de la saga. Appelé une seule fois — les frais ne changent jamais après cette étape.
CommissionStep.java	@Component	Étape 6 — si agent impliqué. Appelle commission-service via REST POST. Calcule la commission agent (0.25%). Crée une entrée DUE dans commission-service. Sa compensation = annulation (DELETE) de cette commission si rollback.
WalletDebitStep.java	@Component	Étape 7 — tous types. Appelle wallet-service via gRPC (WalletService.Debit). Débite le wallet du sender. C'est la première étape financière — si elle réussit et qu'une étape ultérieure échoue, une compensation WALLET_CREDIT est nécessaire.
WalletCreditStep.java	@Component	Étape 8 — tous types. Appelle wallet-service via gRPC (WalletService.Credit). Crédite le wallet du receiver. Si cette étape échoue après que le débit (étape 7) a réussi → rollback immédiat via CompensationService.
LedgerWriteStep.java	@Component	Étape 9 — tous types. Appelle ledger-service via gRPC. Crée les écritures comptables double entrée. Critique pour la conformité BEAC. Compensation = écritures inverses (LEDGER_REVERSAL).
AuditLogStep.java	@Component	Étape 10 — tous types. Publie l'event TransactionCompleted sur Kafka via le pattern Outbox (écriture dans outbox_events). Non bloquant —

Fichier	Type	Rôle dans le Transaction Orchestrator
		même si Kafka est down, l'event est sauvegardé en DB et sera publié plus tard.

executor/compensation/ — Les compensations

Fichier	Type	Rôle dans le Transaction Orchestrator
`WalletCreditCompensation.java`	`@Component`	Rollback de l'étape WALLET_DEBIT. Recredite le sender du montant débité. Utilisé quand une étape après le débit échoue. max_retries=10 car on DOIT réussir — l'argent du client doit être restitué.
`WalletDebitCompensation.java`	`@Component`	Rollback de l'étape WALLET_CREDIT. Débite le receiver du montant crédité. Utilisé dans le scénario FRAUD_BLOCKED où Moussa a déjà reçu l'argent mais la fraude est détectée après.
`LedgerReversalCompensation.java`	`@Component`	Rollback de l'étape LEDGER_WRITE. Crée des écritures comptables inverses dans ledger-service pour annuler l'impact comptable de la transaction. Requis par la BEAC pour l'équilibre des livres.
`CommissionCancelCompensation.java`	`@Component`	Rollback de l'étape COMMISSION_CALC. Annule la commission agent créée si la saga est compensée. Appelle commission-service pour supprimer l'entrée DUE. Évite de payer une commission sur une transaction annulée.

08 Clients — client/

Rôle du dossier client/

Chaque fichier gère la communication avec UN microservice externe.

Les clients REST utilisent RestTemplate ou WebClient. Les clients gRPC utilisent les stubs générés.

Ces classes isolent les détails de communication — si une URL change, on modifie un seul fichier.

client/rest/ — Clients REST

Fichier	Type	Rôle dans le Transaction Orchestrator
`AccountControlClient.java`	`@Component`	Encapsule les appels REST GET vers account-control-service. URL : http://account-control-service/accounts/{id}/status. Gère les timeouts et les erreurs HTTP (404 = compte inexistant, 423 = compte gelé).
`LimitsClient.java`	`@Component`	Encapsule les appels REST POST vers limits-thresholds-service. Envoie le contexte (userId, montant, type) et reçoit OK ou EXCEEDED avec le détail du plafond dépassé.
`KycClient.java`	`@Component`	Encapsule les appels REST GET vers kyc-kyb-orchestrator. Retourne le niveau KYC (KYC1, KYC2, KYC3) et le plafond autorisé. Résultat figé dans kyc_snapshot.
`PinClient.java`	`@Component`	Encapsule les appels REST POST vers pin-service. Envoie le hash du PIN et reçoit VALID ou INVALID. Configure max_retries=1 au niveau du client pour empêcher le bruteforce.
`PricingClient.java`	`@Component`	Encapsule les appels REST POST vers pricing-service. Envoie le type de saga et le montant. Reçoit les frais calculés (fee_rate, fee_amount, rule_id). Résultat figé dans pricing_snapshot.
`CommissionClient.java`	`@Component`	Encapsule les appels REST POST/DELETE vers commission-service. POST = créer une commission DUE. DELETE = annuler une commission lors d'un rollback (CompensationService l'appelle).

Fichier	Type	Rôle dans le Transaction Orchestrator
`AgentRegistryClient.java`	`@Component`	Encapsule les appels REST GET vers agent-registry-service. Vérifie qu'un agent est actif, habilité pour le type d'opération, et que son point de service est opérationnel. Utilisé sur CASH_IN, CASH_OUT, REBALANCING.
`FloatLiquidityClient.java`	`@Component`	Encapsule les appels REST POST vers float-liquidity-service. Vérifie le float disponible (CASH_OUT) et met à jour le float après l'opération. Déclenche une alerte rebalancing si le seuil minimum est atteint.
`CashlogClient.java`	`@Component`	Encapsule les appels REST POST vers cashlog-service. Enregistre le journal de l'opération cash physique : agent, point de service, client, montant, timestamp. Utilisé uniquement sur CASH_IN et CASH_OUT.
`MerchantRegistryClient.java`	`@Component`	Encapsule les appels REST GET vers merchant-registry-service. Vérifie le statut du marchand et récupère le taux de hold reserve applicable. Utilisé uniquement sur P2B.

client/grpc/ — Clients gRPC

Fichier	Type	Rôle dans le Transaction Orchestrator
`WalletGrpcClient.java`	`@Component`	Encapsule les appels gRPC vers wallet-service (WalletService.Debit et WalletService.Credit). Utilisé pour les étapes 7 et 8 — les plus critiques et fréquentes. gRPC choisi pour sa performance (~0.3ms) et son contrat Protobuf strict qui évite les erreurs de format.
`LedgerGrpcClient.java`	`@Component`	Encapsule les appels gRPC vers ledger-service (LedgerService.Write et LedgerService.Reverse). Utilisé pour l'étape 9 (écriture) et la compensation LEDGER_REVERSAL. gRPC choisi pour la même raison que wallet — criticité et fréquence élevées.

09 Kafka — kafka/

Rôle du dossier kafka/

Gère toute la communication asynchrone via Kafka.

Producer : publie les events du pattern Outbox vers les consommateurs.

Consumer : écoute les signaux de fraude pour déclencher une compensation immédiate.

Fichier	Type	Rôle dans le Transaction Orchestrator
`OutboxRelayJob.java`	`@Component`	Job qui tourne toutes les secondes. Lit les outbox_events avec published_at IS NULL via SELECT FOR UPDATE SKIP LOCKED (évite que deux instances publient le même event). Publie sur Kafka et met à jour published_at. C'est le coeur du pattern Outbox — garantit zéro perte d'event.
`FraudSignalConsumer.java`	`@Component`	Consomme le topic mosi.fraud.signals publié par fraud-alerts-service. Si un signal BLOCK arrive pendant qu'une saga est en cours (status=PROCESSING), déclenche immédiatement CompensationService pour annuler les étapes déjà exécutées. C'est le seul endroit où l'orchestrator REÇOIT un event Kafka.

10 Job de recovery — scheduler/

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaRecoveryJob.java`	`@Component`	Job qui tourne toutes les 30 secondes. Détecte les sagas bloquées en PROCESSING (worker crashé) en cherchant updated_at < now()-5min. Reprend l'exécution depuis current_step_order. Détecte aussi les compensations en FAILED avec next_retry_at dépassé et les relance.

Fichier	Type	Rôle dans le Transaction Orchestrator
		C'est le filet de sécurité du système — garantit qu'aucune saga ne reste bloquée indéfiniment.

11 Controller REST — controller/

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaController.java`	`@RestController`	Unique point d'entrée HTTP du microservice. Expose 3 endpoints : POST /sagas (créer une transaction), GET /sagas/{id} (consulter le statut), POST /sagas/{id}/refund (demander un remboursement). Lit le header X-Idempotency-Key et délègue tout à SagaService.

12 DTOs — dto/

Rôle du dossier dto/

Les DTOs sont les objets de transfert de données entre le Controller et le monde extérieur.
Ils ne sont jamais stockés en DB — ce sont juste des enveloppes pour les requêtes et réponses HTTP.
Séparent la structure interne (entités JPA) de ce qu'on expose à l'extérieur.

Fichier	Type	Rôle dans le Transaction Orchestrator
`CreateSagaRequest.java`	`Record/POJO`	Corps de la requête POST /sagas. Contient : sagaType, senderId, receiverId, amountCents, currency, agentId (optionnel), merchantId (optionnel). Validé par @Valid avant d'entrer dans SagaService.
`RefundRequest.java`	`Record/POJO`	Corps de la requête POST /sagas/{id}/refund. Contient : reason (motif de la demande de remboursement). Déclenche la politique de remboursement des 72h.
`SagaResponse.java`	`Record/POJO`	Réponse HTTP après création d'une saga. Contient : sagaId, status, createdAt, refundableUntil. Renvoyé aussi quand une clé d'idempotence est détectée (résultat caché).
`SagaStatusResponse.java`	`Record/POJO`	Réponse HTTP pour GET /sagas/{id}. Contient : sagaId, status, currentStep, totalSteps, errorCode, errorMessage, completedAt. Permet au client de suivre la progression en temps réel.

13 Gestion des erreurs — exception/

Fichier	Type	Rôle dans le Transaction Orchestrator
`SagaNotFoundException.java`	`RuntimeException`	Lancée quand une saga est demandée avec un ID inexistant (GET /sagas/{id}). Le GlobalExceptionHandler la traduit en réponse HTTP 404 avec un message clair.
`IdempotencyConflictException.java`	`RuntimeException`	Lancée si deux requêtes identiques arrivent exactement en même temps et que l'insertion ON CONFLICT DO NOTHING échoue de façon inattendue. Traduite en HTTP 409 Conflict.
`GlobalExceptionHandler.java`	`@RestControllerAdvice`	Intercepte toutes les exceptions non gérées et les traduit en réponses HTTP cohérentes avec un format JSON standardisé : {errorCode, message, timestamp, sagaId}. Évite d'exposer les détails techniques internes.

14 Ressources — resources/

Fichier	Type	Rôle dans le Transaction Orchestrator
application.yml	Config YAML	Configuration principale : port serveur (8080), datasource PostgreSQL, Kafka bootstrap servers, Redis host, paramètres JPA (ddl-auto=validate pour ne pas écraser le schéma Flyway).
application-dev.yml	Config YAML	Surcharge pour l'environnement de développement local : DB locale, logs DEBUG, ddl-auto=create-drop pour repartir de zéro à chaque démarrage.
application-prod.yml	Config YAML	Surcharge pour la production : DB sécurisée (credentials via variables d'environnement), logs WARN uniquement, connexion pool optimisé, timeouts stricts.
V1__create_sagas.sql	Migration Flyway	Crée la table sagas avec toutes ses colonnes, contraintes CHECK (status, saga_type), index (idx_sagas_status, idx_sagas_idempotency_key), et trigger updated_at.
V2__create_saga_steps.sql	Migration Flyway	Crée la table saga_steps avec FK vers sagas (ON DELETE CASCADE), index (idx_steps_saga_id, idx_steps_retry qui filtre les étapes à relancer).
V3__create_saga_audit_events.sql	Migration Flyway	Crée la table saga_audit_events SANS FK intentionnel (rétention BEAC 7 ans). Ajoute les RULE PostgreSQL no_update et no_delete pour l'immuabilité.
V4__create_outbox_events.sql	Migration Flyway	Crée la table outbox_events avec index partiel idx_outbox_pending WHERE published_at IS NULL — optimise la requête du OutboxRelayJob qui ne scanne que les events non publiés.
V5__create_idempotency_keys.sql	Migration Flyway	Crée la table idempotency_keys avec contrainte UNIQUE sur key, index sur expires_at pour la purge automatique des clés expirées.
V6__create_compensation_log.sql	Migration Flyway	Crée la table compensation_log avec contrainte UNIQUE sur idempotency_key, index partiel idx_comp_manual WHERE status = MANUAL_REQUIRED pour le dashboard du Finance Officer.

15 Docker — racine du projet

Fichier	Type	Rôle dans le Transaction Orchestrator
Dockerfile	Docker	Recette pour construire l'image du microservice. Prend le JAR Spring Boot compilé par Maven, le place dans une image Java 21 slim, expose le port 8080. Utilisé pour déployer le microservice en production ou dans docker-compose.
docker-compose.yml	Docker	Lance tout l'environnement local en une commande : transaction-orchestrator (port 8080), PostgreSQL (port 5432), Redis (port 6379), Zookeeper (port 2181), Kafka (port 9092). Indispensable pour développer sans installer chaque service manuellement.
.dockerignore	Docker	Indique à Docker d'ignorer les fichiers inutiles lors de la construction de l'image : target/, .git/, *.md. Réduit la taille de l'image et accélère le build.

16 Tests — test/

Fichier	Type	Rôle dans le Transaction Orchestrator
SagaServiceTest.java	Test JUnit	Teste la logique métier de SagaService : création d'une saga, détection de doublon idempotence, gestion des cas d'erreur. Utilise Mockito pour simuler les appels aux clients REST/gRPC sans appeler les vrais services.

Fichier	Type	Rôle dans le Transaction Orchestrator
`CompensationServiceTest.java`	`Test JUnit`	Teste les scénarios de compensation : rollback partiel, rollback complet, passage en MANUAL_REQUIRED après 10 échecs. Vérifie que l'ordre de compensation est bien l'inverse de l'ordre d'exécution.
`SagaExecutorTest.java`	`Test JUnit`	Teste le moteur d'exécution : étapes dans le bon ordre, retry avec backoff, déclenchement automatique de la compensation sur échec. C'est le test le plus important — il valide tout le comportement du coeur du système.

Synthèse — Tous les fichiers sont en phase avec la tâche

Tâche du Transaction Orchestrator	Fichiers responsables	Pourquoi
Exécuter les étapes d'une saga	SagaExecutor + 10 Steps	Moteur + 1 fichier par étape
Annuler en cas d'échec	CompensationService + 4 Compensations	1 fichier par type de rollback
Protéger contre les doublons	IdempotencyService + IdempotencyKey	Cache Redis + table DB
Tracer pour la BEAC	AuditService + SagaAuditEvent	Journal immuable append-only
Garantir zéro perte Kafka	OutboxRelayJob + OutboxEvent	Pattern Outbox + SKIP LOCKED
Reprendre après un crash	SagaRecoveryJob	Job 30s sur updated_at
Détecter la fraude	FraudSignalConsumer	Consumer Kafka mosi.fraud.signals
Communiquer avec 15 services	10 RestClients + 2 GrpcClients	1 client par service externe
Créer la DB automatiquement	6 migrations Flyway	1 script SQL par table